Проектирование и архитектура многоагентной исследовательской системы ResearchRadar MAS

Коротаев Андрей Дмитриевич

Группа Б24-507

2026

Аннотация

ResearchRadar MAS предназначена для первичного обзора научных публикаций и новостей по исследовательской теме. Система разделяет поиск по типам источников между агентами, сохраняет происхождение записей и журналирует обмен сообщениями. Работа описывает формальную модель, роли, маршруты и реализованный программный срез. Проверки подтвердили воспроизводимый автономный запуск, доступность OpenAlex и резервного RSS-источника. Запросы к GDELT в проверенной среде ограничивались кодом HTTP 429. Содержательный анализ публикаций и проверка новостных утверждений пока не выполняются.

1. Исследовательская задача

Поиск научных публикаций и новостей часто начинается с одного вопроса, но эти источники различаются по структуре, задержке публикации и доказательной силе. Библиографическая запись может содержать DOI и аннотацию, а новость — только заголовок и ссылку. Совместный список без обозначения происхождения создаёт риск принять тематическое сходство за подтверждённую связь. ResearchRadar MAS организует раздельный сбор материалов и возвращает обзор, в котором ограничения каждого источника видны пользователю.

Вход системы — непустая тема на естественном языке. Выход — два набора нормализованных записей, краткая сводка, сведения об ошибках и трасса выполнения. Потребитель результата может открыть исходные URL и продолжить проверку вручную. Текущая версия не извлекает полный текст работ и не формулирует выводы о научных результатах из одних метаданных.

2. Почему используется многоагентная архитектура

Научный и новостной поиск имеют разные источники, форматы данных и ошибки. Их можно выполнять параллельно, не заставляя один компонент знать внутренние правила другого. Координатор отвечает за декомпозицию и назначение задач, поисковые агенты — за работу с собственными источниками, синтезатор — за ограниченное представление собранного результата. Отдельные состояния и очереди позволяют наблюдать, какой участник выполнил действие и где произошёл отказ.

Многоагентность здесь определяется не числом языковых моделей или процессов Windows, а наличием самостоятельных программных сущностей с разными локальными целями, состояниями, полномочиями и сообщениями. В базовом сценарии все агенты работают в одном процессе Python. Их взаимодействие задаётся явным маршрутизатором, а не скрытой функцией агентного фреймворка.

3. Формальная модель

Систему зададим кортежем MAS = (A, E, T, M, R, δ). A — множество экземпляров агентов; E — внешние источники и файловая среда; T — исследовательские задачи; M — структурированные сообщения; R — множество разрешённых направлений передачи; δ — правила изменения локального состояния. Глобальное состояние включает запрос, зарегистрированные задачи, очереди, полученные материалы, ошибки и трассу. Оно формируется из локальных действий агентов и работы маршрутизатора.

Локальное состояние агента s = (phase, handled, last_error) хранит фазу, число обработанных задач и последнюю ошибку. Реализованы переходы idle → busy → done и idle → busy → failed; остальные переходы отклоняются. Цель агента задаёт критерий завершения его задачи, но не

гарантирует качество внешних данных. Система не заявляет доказанных свойств сходимости или оптимального распределения ресурсов.

Роль	Цель и знания	Действия, инструмент и полномочия
Координатор	Создать подзадачи и собрать ответы; знает исходный запрос, компетенции и сообщения.	Выбирает исполнителей, направляет задачи, принимает результаты и ошибки; не выполняет поиск сам.
Научный агент	Получить библиографические записи; знает свою задачу и ответ источника.	Вызывает OpenAlex или автономный адаптер, нормализует публикации и отвечает координатору.
Новостной агент	Получить новостные записи; знает свою задачу и ответ источника.	Вызывает GDELT, RSS или автономный адаптер, нормализует новости и отвечает координатору.
Синтезатор	Подвести ограниченную сводку; знает оба набора записей и их объём.	Считает записи, указывает границы доступного текста; не устанавливает доказательные связи.

Роль описывает функцию и полномочия, экземпляр — конкретный объект с собственной очередью. Сейчас каждой роли соответствует один работающий экземпляр. Реестр компетенций допускает регистрацию дополнительных поисковых исполнителей с пересекающимися возможностями; базовая политика выбирает первого доступного кандидата. Изменение политики не требует переписывания контрактов сообщений или адаптеров.

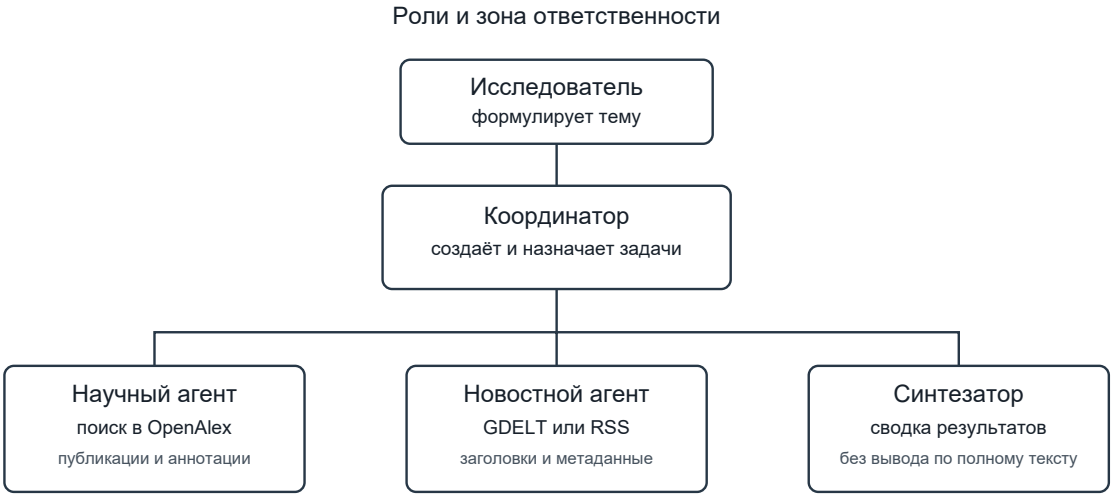


Рисунок 1 — Организация ролей и ответственности

4. Коммуникация и обработка задач

Сообщение содержит уникальный id, отправителя, получателя, тип, идентификатор исследования, полезную нагрузку и correlation_id ответа. Маршрутизатор проверяет разрешённый путь перед постановкой сообщения в личную очередь адресата и записывает событие в трассу. Координатор отправляет task поисковым агентам; они возвращают result либо error. Затем координатор отправляет task синтезатору и получает result. Прямой обмен между поисковыми агентами запрещён.

Научная и новостная подзадачи выполняются конкурентно через asyncio. Вызовы HTTP вынесены в рабочие потоки, чтобы не блокировать цикл событий. Координатор собирает ответы, сопоставляя их с назначенными исполнителями и идентификатором исследования. Общий срок ожидания ограничен; незавершённые асинхронные задачи отменяются. Если один источник отказал, ответ другого остаётся в результате, а ошибка сохраняется отдельно.



Рисунок 2 — Разрешённые направления структурированных сообщений

5. Данные, источники и границы интерпретации

Объект материала хранит тип записи, название источника, его первичный идентификатор, заголовок, URL, дату, DOI при наличии, аннотацию, объём доступного текста и происхождение. OpenAlex нормализуется как научный источник; GDELT и Google News RSS — как новостные. Записи без заголовка или URL не включаются. Дубликаты удаляются только по паре «источник, первичный идентификатор»: приблизительное смысловое сходство не означает тождество.

Автономный режим возвращает явно маркированные синтетические записи. В сетевом режиме GDELT делает не более одного повторного запроса после HTTP 429, учитывает Retry-After и сохраняет успешный ответ в локальном кэше с ограниченным сроком жизни. RSS выбирается явно, поэтому успешность резервного источника не маскирует отказ GDELT. Программная сводка не делает выводов о методах, результатах исследований или достоверности новостей по одним заголовкам.

Клиент Qwen через OpenRouter отделён от основного исследовательского цикла. Он сохраняет запись в SQLite до отправки платного запроса, резервирует оценочную стоимость, обновляет токены и фактическую стоимость после ответа, отмечает неопределённый расход при таймауте и блокирует повтор с тем же ID. Текущая версия использует этот клиент для отдельной проверки доступности модели; интеллектуальный анализ в основном цикле пока не заявляется.

6. Проверка реализации

Проверка	Наблюдение
Автономный запуск	Две синтетические записи разных типов, шесть сообщений в трассе; платных вызовов нет.
Автоматические тесты	20 тестов на Python 3.14: состояния, маршруты, нормализация, отказ источника, CLI, GDELT и журнал расходов.
OpenAlex	Один сетевой запрос по теме artificial intelligence вернул три научные записи.
GDELT	Один аккуратный запрос получил HTTP 429. Успешность этого адаптера в текущей среде не подтверждена.
Google News RSS	Явный резервный режим вернул три новостные записи по той же теме.
Qwen	Отдельный вызов вернул ответ; журнал сохранил ID провайдера, 16 входных и 154 выходных токена, фактическую стоимость 0,0001564 USD.

Предыдущая серия проверок включала два учтённых вызова OpenRouter на 0,0002548 USD и один короткий вызов, точная стоимость которого не сохранилась. Эти исторические данные не суммируются с новым журналом как полный подтверждённый расход. Исправленный механизм учёта проверен тестами без реальных запросов на каждом сценарии.

7. Ограничения и направления развития

Нынешняя сводка перечисляет материалы и обозначает пределы данных; она не анализирует полный текст, не проверяет утверждения и не устанавливает связь между конкретной публикацией и конкретной новостью. Координатор остаётся единой точкой отказа. Трасса фиксирует маршруты и время сообщений, но не полные снимки локальных состояний. Внешние API могут возвращать неполные записи, ограничивать частоту запросов или менять формат ответа.

Разделение компетенций и явный реестр кандидатов позволяют в дальнейшем сравнить политики назначения задач на одинаковом наборе запросов. Для исследования конкуренции за ограниченный ресурс потребуется отдельный механизм заявок и учёта справедливости. Для испытаний устойчивости следует расширить трассу снимками состояний и добавить контролируемое внедрение ошибок. Эти возможности описаны как направления развития, а не как уже проверенные свойства.

Заключение

ResearchRadar MAS реализует небольшой, наблюдаемый многоагентный цикл: декомпозицию запроса, конкурентный поиск по разным типам источников, структурированный обмен сообщениями и ограниченную сводку. Архитектурные роли соответствуют программным экземплярам, а границы полномочий проверяются маршрутизатором. Автономный сценарий и резервный RSS-поиск подтверждены; ограничение GDELT и неполнота доступных текстов остаются существенными условиями применения результата.